

TEST FOR ANYLOGISTIX API

The API info is here: <https://anylogistix.help/api/python/readme.html>

PHASE 1

Install the dependencies

It assumes that the openapi_client is in the local directory ./openapi-python-client-3.3.1

```
In [38]: import sys
import os
# This is my personal directory
os.chdir(r"C:\Users\Luis\Downloads\TFG\API")
#!{sys.executable} -m pip install "pydantic>=2.10,<3"
#!{sys.executable} -m pip install "pydantic<2,>=1.10.5"
#!{sys.executable} -m pip install -e ./openapi-python-client-3.3.1
```

```

Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (1.10.26)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from pydantic<2,>=1.10.5) (4.15.0)
Obtaining file:///C:/Users/Luis/Downloads/TFG/API/openapi-python-client-3.3.1
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Checking if build backend supports build_editable: started
  Checking if build backend supports build_editable: finished with status 'done'
  Getting requirements to build editable: started
  Getting requirements to build editable: finished with status 'done'
  Preparing editable metadata (pyproject.toml): started
  Preparing editable metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: urllib3<2.1.0,>=1.25.3 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from openapi-client==1.0.0) (2.0.7)
Requirement already satisfied: python-dateutil in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from openapi-client==1.0.0) (2.9.0.post0)
Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from openapi-client==1.0.0) (1.10.26)
Requirement already satisfied: aenum in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from openapi-client==1.0.0) (3.1.16)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from pydantic<2,>=1.10.5->openapi-client==1.0.0) (4.15.0)
Requirement already satisfied: six>=1.5 in c:\users\luis\downloads\tfg\anaconda\lib\site-packages (from python-dateutil->openapi-client==1.0.0) (1.17.0)
Building wheels for collected packages: openapi-client
  Building editable for openapi-client (pyproject.toml): started
  Building editable for openapi-client (pyproject.toml): finished with status 'done'
  Created wheel for openapi-client: filename=openapi_client-1.0.0-0.editable-py3-none-any.whl size=3042 sha256=4c9a2cbd6800668c211860effd77646b8b249e51a2c3b3475165635c8d9ab544
  Stored in directory: C:\Users\Luis\AppData\Local\Temp\pip-ephem-wheel-cache-7acga0ay\wheels\c8\d9\98\fb47f5f4ed4f17d5eaf5c2b82379b16ecfb0fe485c310ce133
Successfully built openapi-client
Installing collected packages: openapi-client
  Attempting uninstall: openapi-client
    Found existing installation: openapi-client 1.0.0
    Uninstalling openapi-client-1.0.0:
      Successfully uninstalled openapi-client-1.0.0
Successfully installed openapi-client-1.0.0

```

```

In [1]: import openapi_client
        from openapi_client.rest import ApiException
        from pprint import pprint
        import urllib3

```

```

In [2]: # SERVER_IP="192.168.64.159"
        # SERVER_IP="192.168.67.110"
        SERVER_IP="alxserver.aut.uah.es"
        SERVER_URL=f"https://{SERVER_IP}:443/api/v1"

        # This is my personal APY KEY
        API_KEY="c184f1ab-9f13-484c-a1c1-3d543502da6e"

        SERVER_URL

```

```

Out[2]: 'https://alxserver.aut.uah.es:443/api/v1'

```

PART I: CHECK CONNECTIVITY AND OPERATIONAL STATUS

```
In [3]: # Defining the host is optional and defaults to https://server_address:port/api/
# See configuration.py for a list of all supported configuration parameters.
configuration = openapi_client.Configuration(
    host = SERVER_URL
)
# The client must configure the authentication and authorization parameters
# in accordance with the API server security policy.
# Examples for each auth method are provided below, use the example that
# satisfies your auth use case.
# Configure API key authorization: ApiKey
configuration.api_key['ApiKey'] = API_KEY

# ALVARO: SUPER IMPORTANTE PARA PODER TRABAJAR CON CERTIFICADOS AUTO-FIRMADOS
configuration.verify_ssl = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
with openapi_client.ApiClient(configuration) as api_client:
    # Create an instance of the API class
    api_instance = openapi_client.OpenApiApi(api_client)
```

Obtain current user data

```
In [4]: try:
# Gets current user data.
api_response = api_instance.get_current_user()
print("The response of OpenApiApi->get_current_user:\n")
pprint(api_response)
except Exception as e:
print("Exception when calling OpenApiApi->get_current_user: %s\n" % e)
```

The response of OpenApiApi->get_current_user:

```
ApiUserData(email='adrian.encabo@edu.uah.es', first_name='adrian.encabo@edu.uah.e
s', id=403, last_name='', username='adrian.encabo@edu.uah.es')
```

PART II: OBTAIN PROJECTS, SCENARIOS, EXECUTIONS

Obtain the list of currently available projects

```
In [5]: try:
# Returns a List of projects that the user has access to.
api_response = api_instance.get_projects()
print("The response of OpenApiApi->get_projects:\n")
pprint(api_response)
except Exception as e:
print("Exception when calling OpenApiApi->get_projects: %s\n" % e)
```

The response of OpenApiApi->get_projects:

```
[ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)]
```

Open a project and get the list of scenarios available

```
In [19]: # My personal Project defined
MY_PROJECT_NAME="TFG_ADRIAN_ENCABO"
the_project_id=0
try:
    full_match = True # bool | Whether to match project name by complete match.
    # Opens project with name projectName.
    the_project = api_instance.find_and_open_project_by_name(full_match, MY_PROJECT_NAME)
    print("The response of OpenApiApi->find_and_open_project_by_name:\n")
    pprint(the_project)
except Exception as e:
    print("Exception when calling OpenApiApi->find_and_open_project_by_name: %s\n" % e)

try:
    # Returns a List of scenarios for the project with identifier project_id.
    the_scenarios = api_instance.get_scenarios(the_project.id)
    print("The response of OpenApiApi->get_scenarios:\n")
    pprint(the_scenarios)
except Exception as e:
    print("Exception when calling OpenApiApi->get_scenarios: %s\n" % e)
```

The response of OpenApiApi->find_and_open_project_by_name:

```
ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)
```

The response of OpenApiApi->get_scenarios:

```
[ApiScenarioData(id=82840, name='Budget Comparison (Estimated Demand)', type='SIM'),
 ApiScenarioData(id=87083, name='P3.3. GFA 1. Results 2_Different Locations', type='SIM'),
 ApiScenarioData(id=84191, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=85538, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=86671, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction', type='NO'),
 ApiScenarioData(id=88185, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction 2', type='NO'),
 ApiScenarioData(id=89212, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction sim', type='SIM')]
```

Select the scenario by name

```
In [20]: #Add the name of the scenario
MY_SCENARIO_NAME="Budget Comparison (Estimated Demand)"

# 1. Select the scenario
the_scenario = next(s for s in the_scenarios if s.name == MY_SCENARIO_NAME)
pprint(the_scenario)
```

```
ApiScenarioData(id=82840, name='Budget Comparison (Estimated Demand)', type='SIMULATION')
```

Get the list of experiment runs of a scenario

```
In [21]: try:
# Returns the results of experiment runs for scenario.
the_runs = api_instance.get_experiment_runs_for_scenario(the_scenario.id)
print("The response of OpenApiApi->get_experiment_runs_for_scenario:\n")
pprint(the_runs)
except Exception as e:
print("Exception when calling OpenApiApi->get_experiment_runs_for_scenario:
```

The response of OpenApiApi->get_experiment_runs_for_scenario:

```
[ApiBasicRun(has_options=False, id=91271, name='Statistics', rc_id=91272, removing=False, scenario_id=82840, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=91536, name='Statistics 2', rc_id=91537, removing=False, scenario_id=82840, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=91770, name='Statistics 3', rc_id=91771, removing=False, scenario_id=82840, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=91999, name='Statistics 4', rc_id=92000, removing=False, scenario_id=82840, type='SIMULATION')]
```

PART III. EXECUTE A SIMULATION

Get the list of RunConfigurations for the experiments of a given Scenario

Each type of experiment has a RunConfiguration. For example:

```
ApiBasicRunConfiguration(animation_model_id=None,
feasible_check_status='NOT_CONDUCTED', id=2439,
is_feasible_check_data_available=False, model_time=None,
name='i18n-Experiment_Simulation_text', progress=None,
project_id=63, routes_progress=None, run_status=None,
scenario_id=1752, speed=None, status='IDLE', step='IDLE',
type='SIMULATION', validation_status='VALID')
```

Types of available experiment runs are:

- SIMULATION
- VARIATION
- COMPARISON
- SAFETY_STOCK
- RISK_ANALYSIS

```
In [22]: MY_RUN_TYPE="SIMULATION"
try:
# Returns a List of experiments available for a given scenario.
the_run_configurations= api_instance.get_experiments(the_project.id, the_scenario_id)
print("The response of OpenApiApi->get_experiments:\n")
# Exception fixed for undefined variable
pprint(the_run_configurations)
```

```
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiments: %s\n" % e)

# 2. Select the adequate RunConfiguration for the scenario
the_RC = next((r for r in the_run_configurations if r.type == MY_RUN_TYPE), None)
pprint(the_RC)
```

The response of OpenApiApi->get_experiments:

```
[ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=83528, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=83698, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_VariationExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='VARIATION', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=82848, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_ComparisonExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='COMPARISON', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=83389, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_SafetyStockEstimation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='SAFETY_STOCK', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=83204, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_RiskAnalysisExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='RISK_ANALYSIS', validation_status='NOT_CONDUCTED')]
ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=83528, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=82840, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID')
```

Run a new experiment over a scenario (synch)

Runs synchronously.

```
In [23]: try:
# Starts the experiment synchronously.
the_sync_result = api_instance.run_experiment_synchronously(the_RC.id)
print("The response of OpenApiApi->run_experiment_synchronously:\n")
pprint(the_sync_result)
except Exception as e:
    print("Exception when calling OpenApiApi->run_experiment_synchronously: %s\n"
```

The response of OpenApiApi->run_experiment_synchronously:

```
ApiExperimentResult(experiment_result_id=92205, validation_errors=None, validation_status='OK', validation_warnings=None)
```

Part IIIb. Asynch execution

Run a new experiment over a scenario (asynch)

Runs asynchronously. Periodic status check.

```
In [24]: skip_scenario_warnings = True # bool / Whether to skip scenario warnings. (optional)
try:
    # Starts the experiment asynchronously.
    the_execution = api_instance.run_experiment(the_RC.id, skip_scenario_warning
    print("The response of OpenApiApi->run_experiment:\n")
    pprint(the_execution)
except Exception as e:
    print("Exception when calling OpenApiApi->run_experiment: %s\n" % e)

import time
while True:
    try:
        # Returns the experiment status
        the_status = api_instance.get_experiment_status(the_RC.id)
        # pprint(api_response)

        # Check if execution finished
        if the_status.finished:
            print("Execution finished.")
            break

        # Optional: stop if failed
        # fixed exception of status check
        if the_status.failed:
            print("Execution failed.")
            break

    except Exception as e:
        print("Exception when calling OpenApiApi->get_experiment_status:", e)
        print("Not finished yet. Wait 1sec and check again")
        # wait before next check
        time.sleep(1)
```

The response of OpenApiApi->run_experiment:

```
ApiRunExperiment(animation_model_id=83528, experiment_type_id='SIMULATION', is3d_
animation=False)
Not finished yet. Wait 1sec and check again
Not finished yet. Wait 1sec and check again
Not finished yet. Wait 1sec and check again
Execution finished.
```

Get the experiments results (pages)

You can ask for limited number of results (avoid overflow)

```
In [25]: skip = 0 # int / Number of records to skip. (default to 0)
take = 1000 # int / Number of records to be retrieved. (default to 1000)

try:
    # Returns all the results of the specific experiment run.
    the_run_results = api_instance.get_experiment_run_result(the_RC.id, skip, ta
    print("The response of OpenApiApi->get_experiment_run_result:\n")
```

```
pprint(the_run_results)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_run_result: %s\n" %
```

The response of OpenApiApi->get_experiment_run_result:

```
ApiExperimentResultData(pages=[ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83531, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Revenue, Total Cost', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83531, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=83532, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Profit and Loss Statement', rc_id=83528, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=83532, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83533, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Profit, Revenue, Total Cost', rc_id=83528, type='BAR'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83533, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR'))], page=ApiPage(id='83530', name='Profit and Loss Statement')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83535, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Service Level by Products (Per Product)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83535, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83536, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Service Level by Products (Per Source)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83536, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83537, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Service Level by Revenue (Per Object)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83537, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83538, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='ELT Service Level by Products (Per Product)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83538, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83539, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='ELT Service Level by Products (Per Source)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83539, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83540, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='ELT Service Level by Revenue (Per Object)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83540, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='83534', name='Service Level')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83543, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Lead Time', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83543, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83544, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Mean Lead Time (Best-Mean-Worst
```

```

t)', rc_id=83528, type='BMW-LINE'), class_name='ApiExperimentResultGraphChartWrap
per', data=ApiGraphChartData(entity_list=[], id=83544, show_x=True, total_entity_
count=0, total_technical_row_count=0, type='BMW-LINE')), ApiExperimentResultTable
ChartWrapper(chart=ApiChartMetadataShort(id=83545, is_group_mode_enabled=False, l
ayout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Max
Lead Time', rc_id=83528, type='TABLE'), class_name='ApiExperimentResultTableChart
Wrapper', data=ApiStrictTableChartData(entity_list=[], id=83545, total_entity_cou
nt=0, total_technical_row_count=0, type='TABLE'))], page=ApiPage(id='83542', name
='Lead Time')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphCh
artWrapper(chart=ApiChartMetadataShort(id=83547, is_group_mode_enabled=False, lay
out=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Availa
ble Inventory', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphCh
artWrapper', data=ApiGraphChartData(entity_list=[], id=83547, show_x=True, total_
entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGr
aphChartWrapper(chart=ApiChartMetadataShort(id=83548, is_group_mode_enabled=False,
layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name
='On-hand Inventory', rc_id=83528, type='LINE'), class_name='ApiExperimentResultG
raphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83548, show_x=True,
total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentRe
sultGraphChartWrapper(chart=ApiChartMetadataShort(id=83549, is_group_mode_enabled
=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), n
ame='Average Daily Available Inventory', rc_id=83528, type='LINE'), class_name='A
piExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=8
3549, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LIN
E')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83550,
is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, sta
rt_row=1, width=2), name='Available Inventory Including Backlog', rc_id=83528, ty
pe='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChart
Data(entity_list=[], id=83550, show_x=True, total_entity_count=0, total_technical
_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartM
etadataShort(id=83551, is_group_mode_enabled=False, layout=ApiChartLayoutData(hei
ght=1, start_col=2, start_row=1, width=2), name='Average Daily On-hand Inventor
y', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper',
data=ApiGraphChartData(entity_list=[], id=83551, show_x=True, total_entity_count=
0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='83546', name='Av
ailable Inventory')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultT
ableChartWrapper(chart=ApiChartMetadataShort(id=83553, is_group_mode_enabled=False,
layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=3), name
='Peak Capacity', rc_id=83528, type='TABLE'), class_name='ApiExperimentResultTabl
eChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=83553, total_enti
ty_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraph
ChartWrapper(chart=ApiChartMetadataShort(id=83554, is_group_mode_enabled=False, l
ayout=ApiChartLayoutData(height=1, start_col=3, start_row=0, width=3), name='Peak
Capacity', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWr
apper', data=ApiGraphChartData(entity_list=[], id=83554, show_x=True, total_entit
y_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='83552',
name='Peak Capacity')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResul
tTableChartWrapper(chart=ApiChartMetadataShort(id=83556, is_group_mode_enabled=False,
layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name
='Demand Placed, Fulfillment Received (Accumulative)', rc_id=83528, type='TABL
E'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartD
ata(entity_list=[], id=83556, total_entity_count=0, total_technical_row_count=0,
type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort
(id=83557, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start
_col=2, start_row=0, width=2), name='Demand Placed, Fulfillment Received (Dail
y)', rc_id=83528, type='LINE'), class_name='ApiExperimentResultGraphChartWrappe
r', data=ApiGraphChartData(entity_list=[], id=83557, show_x=True, total_entity_co
unt=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultTableChartW
rapper(chart=ApiChartMetadataShort(id=83558, is_group_mode_enabled=False, layout=
ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Demand Rec

```

```
eived, Fulfillment Shipped (Accumulative)', rc_id=83528, type='TABLE')), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=83558, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=83559, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Accumulative, Per Object)', rc_id=83528, type='TABLE')), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=83559, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83560, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Daily, Per Object)', rc_id=83528, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83560, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83561, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='Demand Received, Fulfillment Shipped (Daily)', rc_id=83528, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83561, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='83555', name='Fulfillment')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=83563, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Max Number of Vehicles Used', rc_id=83528, type='TABLE')), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=83563, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83564, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Shipped Vehicles', rc_id=83528, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83564, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=83565, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Average Number of Vehicles Used', rc_id=83528, type='BAR')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=83565, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR'))], page=ApiPage(id='83562', name='Used Vehicles')))]
```

```
In [30]: for p in the_run_results.pages:
          print(f"\n{p.page.name} (id:{p.page.id})")
          for c in p.charts:
              print(f"    - {c.chart.name} (type: {c.chart.type}) (id: {c.chart.id})")
```

Profit and Loss Statement (id:83530)

- Revenue, Total Cost (type: LINE) (id: 83531)
- Profit and Loss Statement (type: TABLE) (id: 83532)
- Profit, Revenue, Total Cost (type: BAR) (id: 83533)

Service Level (id:83534)

- Service Level by Products (Per Product) (type: LINE) (id: 83535)
- Service Level by Products (Per Source) (type: LINE) (id: 83536)
- Service Level by Revenue (Per Object) (type: LINE) (id: 83537)
- ELT Service Level by Products (Per Product) (type: LINE) (id: 83538)
- ELT Service Level by Products (Per Source) (type: LINE) (id: 83539)
- ELT Service Level by Revenue (Per Object) (type: LINE) (id: 83540)

Lead Time (id:83542)

- Lead Time (type: LINE) (id: 83543)
- Mean Lead Time (Best-Mean-Worst) (type: BMW-LINE) (id: 83544)
- Max Lead Time (type: TABLE) (id: 83545)

Available Inventory (id:83546)

- Available Inventory (type: LINE) (id: 83547)
- On-hand Inventory (type: LINE) (id: 83548)
- Average Daily Available Inventory (type: LINE) (id: 83549)
- Available Inventory Including Backlog (type: LINE) (id: 83550)
- Average Daily On-hand Inventory (type: LINE) (id: 83551)

Peak Capacity (id:83552)

- Peak Capacity (type: TABLE) (id: 83553)
- Peak Capacity (type: LINE) (id: 83554)

Fulfillment (id:83555)

- Demand Placed, Fulfillment Received (Accumulative) (type: TABLE) (id: 83556)
- Demand Placed, Fulfillment Received (Daily) (type: LINE) (id: 83557)
- Demand Received, Fulfillment Shipped (Accumulative) (type: TABLE) (id: 83558)
- Demand Placed, Fulfillment Received (Accumulative, Per Object) (type: TABLE) (id: 83559)
- Demand Placed, Fulfillment Received (Daily, Per Object) (type: LINE) (id: 83560)
- Demand Received, Fulfillment Shipped (Daily) (type: LINE) (id: 83561)

Used Vehicles (id:83562)

- Max Number of Vehicles Used (type: TABLE) (id: 83563)
- Shipped Vehicles (type: LINE) (id: 83564)
- Average Number of Vehicles Used (type: BAR) (id: 83565)

Obtain the pages in the experiment's result dashboard

```
In [35]: try:
# Returns statistics pages for the result of experiment run.
# Exception fixed for 'id' attribute
# We use the experiment_result_id from the previously executed synchronous r
result_id = the_sync_result.experiment_result_id
dashboard_pages = api_instance.get_experiment_dashboard_pages(result_id)
print("The response of OpenApiApi->get_experiment_dashboard_pages:\n")

# CHANGED Displaying the available pages and charts with their dynamic IDs
for page in dashboard_pages:
    print(f"Dashboard Page: {page.name} (ID: {page.id})")
    for chart in page.charts:
        print(f" - Chart: {chart.name} (ID: {chart.id})")
```

```
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_dashboard_pages: %s"
```

The response of OpenApiApi->get_experiment_dashboard_pages:

Dashboard Page: Profit and Loss Statement (ID: 92207)

- Chart: Revenue, Total Cost (ID: 92208)
- Chart: Profit and Loss Statement (ID: 92209)
- Chart: Profit, Revenue, Total Cost (ID: 92210)

Dashboard Page: Service Level (ID: 92211)

- Chart: Service Level by Products (Per Product) (ID: 92212)
- Chart: Service Level by Products (Per Source) (ID: 92213)
- Chart: Service Level by Revenue (Per Object) (ID: 92214)
- Chart: ELT Service Level by Products (Per Product) (ID: 92215)
- Chart: ELT Service Level by Products (Per Source) (ID: 92216)
- Chart: ELT Service Level by Revenue (Per Object) (ID: 92217)

Dashboard Page: Lead Time (ID: 92219)

- Chart: Lead Time (ID: 92220)
- Chart: Mean Lead Time (Best-Mean-Worst) (ID: 92221)
- Chart: Max Lead Time (ID: 92222)

Dashboard Page: Available Inventory (ID: 92223)

- Chart: Available Inventory (ID: 92224)
- Chart: On-hand Inventory (ID: 92225)
- Chart: Average Daily Available Inventory (ID: 92226)
- Chart: Available Inventory Including Backlog (ID: 92227)
- Chart: Average Daily On-hand Inventory (ID: 92228)

Dashboard Page: Peak Capacity (ID: 92229)

- Chart: Peak Capacity (ID: 92230)
- Chart: Peak Capacity (ID: 92231)

Dashboard Page: Fulfillment (ID: 92232)

- Chart: Demand Placed, Fulfillment Received (Accumulative) (ID: 92233)
- Chart: Demand Placed, Fulfillment Received (Daily) (ID: 92234)
- Chart: Demand Received, Fulfillment Shipped (Accumulative) (ID: 92235)
- Chart: Demand Placed, Fulfillment Received (Accumulative, Per Object) (ID: 92236)
- Chart: Demand Placed, Fulfillment Received (Daily, Per Object) (ID: 92237)
- Chart: Demand Received, Fulfillment Shipped (Daily) (ID: 92238)

Dashboard Page: Used Vehicles (ID: 92239)

- Chart: Max Number of Vehicles Used (ID: 92240)
- Chart: Shipped Vehicles (ID: 92241)
- Chart: Average Number of Vehicles Used (ID: 92242)

Export the results as an excel matrix

```
In [40]: # Exception fixed for wrong IDs
import shutil #For manipulating excel files
try:
    # Dynamically select the first dashboard page to export to avoid hardcoded ID
    if dashboard_pages and len(dashboard_pages) > 0:
        target_page_id = dashboard_pages[0].id
        target_result_id = the_sync_result.experiment_result_id

    print(f"Exporting dashboard page ID {target_page_id} from result ID {target_result_id}")

    # Returns an Excel representation of the dashboard page
    excel_export = api_instance.export_dashboard_page(target_result_id, target_page_id)

    # Save the exported Excel file locally
```

```

output_filename = "Simulation_Results.xlsx"

# Check if the API returns a temporary file path
if isinstance(excel_export, str) and os.path.exists(excel_export):
    shutil.move(excel_export, output_filename)
    print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

# Check if the API returns binary data directly
elif isinstance(excel_export, bytes):
    with open(output_filename, "wb") as file:
        file.write(excel_export)
    print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

else:
    print(f"Unexpected data format received. Type: {type(excel_export)}")
    pprint(excel_export)

print("\nData exported successfully.")
else:
    print("No dashboard pages available to export.")

except Exception as e:
    print("Exception when calling OpenApiApi->export_dashboard_page: %s\n" % e)

```

Exporting dashboard page ID 92207 from result ID 92205...

Excel file saved successfully to: C:\Users\Luis\Downloads\TFG\API\Simulation_Results.xlsx

Data exported successfully.

Close current project

In [39]: *#Error fixed of indentation*

```

try:
    # Closes the currently open project.
    api_response = api_instance.close_project()
    print("The response of OpenApiApi->close_project:\n")
    pprint(api_response)
    print("\nProject closed successfully.")
except ApiException as e:
    print("Exception when calling OpenApiApi->close_project: %s\n" % e)

```

The response of OpenApiApi->close_project:

```

ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=None, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)

```

Project closed successfully.

=====

PHASE 2

◀  ▶

In []:

=====

DEVELOPMENT TESTS - IGNORE THEM

◀ —▶

```
In [ ]: #Avoid self-signed warnings
import os
import urllib3
```

For self-signed certificates it is necessary to download them and add to the certification path

```
In [ ]: import ssl
import socket

def download_self_signed_cert_no_validation(hostname, port, cert_file):
    context = ssl._create_unverified_context()
    conn = context.wrap_socket(socket.socket(socket.AF_INET), server_hostname=hostname)
    conn.connect((hostname, port))
    der_cert = conn.getpeercert(binary_form=True)
    pem_cert = ssl.DER_cert_to_PEM_cert(der_cert)

    with open(cert_file, 'w') as cert_file:
        cert_file.write(pem_cert)

    print(f"Certificate saved to {cert_file.name}")

# Example usage
download_self_signed_cert_no_validation(SERVER_IP, 443, MY_CERT_FILENAME)
```

```
In [ ]: # !/usr/bin/openssl s_client -showcerts -connect {SERVER_IP}:443 </dev/null 2>/dev/null
```

```
In [ ]: MY_CERT_FILE=os.path.abspath(MY_CERT_FILENAME)
if os.path.isfile(MY_CERT_FILE):
    print(f"Cert file OK: {MY_CERT_FILE} ")
else:
    print(f"Cert file ERROR: {MY_CERT_FILE} ")
MY_CERT_FILE
```

```
In [ ]: os.environ['REQUESTS_CA_BUNDLE'] = MY_CERT_FILE

# Create a PoolManager with a custom CA bundle
http = urllib3.PoolManager(
    cert_reqs='CERT_NONE',      # Enforce cert verification
    ca_certs=MY_CERT_FILE      # Path to your self-signed cert
)

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
http.request("GET", TEST_URL)
```

```
In [ ]: from urllib3 import HTTPSConnectionPool

pool = HTTPSConnectionPool(
    SERVER_IP,
    port=443,
```

```
cert_reqs='CERT_REQUIRED',  
ca_certs=MY_CERT_FILE  
)
```

```
In [ ]: import requests  
  
# Any request you make now will use your certificate  
response = requests.get(TEST_URL, verify=False)  
print(response.text)
```

```
In [ ]:
```